

hayplot

Plugin de Visualização Declarativa para Hayashi

Grammar of Graphics para a linguagem Hayashi

Baseado em Apache Arrow FFI e Plotters (Rust puro)

Versão hayplot	v1.3.0
Versão Hayashi	0.2.6
Repositório	sheep-farm/hayplot
Licença	MIT

Sumário

1	Introdução	3
1.1	Filosofia	3
1.2	Características Técnicas	3
1.3	Instalação	4
1.4	Importando o Plugin	4
2	Referência das Funções	5
2.1	hayplot(df, aes) — Inicialização	5
2.2	geom_point(plot, color, size) — Camada de Pontos	6
2.3	geom_line(plot, color, size) — Camada de Linha	7
2.4	geom_bar(plot, color, width) — Camada de Barras	7
2.5	geom_histogram(plot, color, bins) — Camada de Histograma	8
2.6	geom_boxplot(plot, color, width) — Camada de Boxplot	9
2.7	geom_heatmap(plot, color, cell_size) — Camada de Heatmap	10
2.8	geom_area(plot, color, size) — Camada de Área	11
2.9	geom_hline(plot, color, size, yintercept) — Linha Horizontal	12
2.10	geom_vline(plot, color, size, xintercept) — Linha Vertical	13
2.11	geom_abline(plot, color, size, slope, intercept) — Linha Diagonal	14
2.12	geom_step(plot, color, size, direction) — Linha em Degrau	14
2.13	scale_x_log10(plot) — Escala Logarítmica em X	15
2.14	scale_y_log10(plot) — Escala Logarítmica em Y	16
2.15	filter_data(df, col, value) — Filtragem de Dados	16
2.16	set_dimensions(plot, width, height) — Dimensões do SVG	17
2.17	set_margins(plot, top, bottom, left, right) — Margens do Plot	18
2.18	save_svg(plot, filename) — Salvar SVG	19
2.19	set_background_color(plot, color) — Cor de Fundo	19
2.20	set_grid(plot, show_grid) — Controle do Grid	20
2.21	save_png(plot, filename) — Salvar PNG	20
2.22	labs(plot, title, x, y) — Rótulos	21
2.23	render_svg(plot) — Renderização	22
3	Fluxo de Trabalho	24
3.1	Pipeline Declarativo	24
3.2	Tipos de Colunas Suportados	24
3.3	Valores Ausentes	25

4	Exemplos Completos	26
4.1	Dispersão Simples	26
4.2	Curva de Phillips	26
4.3	A partir de Arquivo CSV	27
4.4	Combinando com Regressão OLS	28
4.5	Pipeline Multi-linha	28
4.6	Linha com Marcadores de Pontos	29
4.7	Gráfico de Barras	29
4.8	Histograma	30
4.9	Boxplot	30
4.10	Heatmap	31
4.11	Gráfico de Área	31
4.12	Linhas de Referência	32
4.13	Gráfico de Degrau	32
4.14	Escala Logarítmica	33
4.15	Faceting Manual	33
5	Arquitetura Técnica	35
5.1	Comunicação via FFI Arrow	35
5.2	Dependências	35
5.3	Compilação e Distribuição	36
6	Guia de Contribuição	37
6.1	Configurando o Ambiente	37
6.2	Testando com o Hayashi Local	37
6.3	Adicionando Novas Geometrias	37
7	Roadmap	39
8	Referência Rápida	40

Capítulo 1

Introdução

hayplot é um plugin nativo para a linguagem Hayashi que implementa uma **Gramática de Gráficos** (*Grammar of Graphics*) ao estilo do **ggplot2** do R. Ele permite construir visualizações de dados de forma declarativa e composicional, usando o operador de pipe `|>` da linguagem.

1.1 Filosofia

A ideia central é que um gráfico pode ser descrito como a composição de *camadas semânticas independentes*:

1. **Dados** — o `DataFrame` de entrada;
2. **Mapeamento estético** (*aes*) — quais colunas mapeiam para x e y ;
3. **Geometrias** (*geoms*) — como os dados serão representados visualmente;
4. **Rótulos** (*labs*) — título e nomes dos eixos.

Cada camada é uma transformação pura de um dicionário de especificação, e a renderização final acontece apenas ao chamar `render_svg()`.

1.2 Características Técnicas

- **Zero-Copy**: os dados do `DataFrame` são compartilhados com o plugin via Apache Arrow FFI sem cópia ou serialização — o ponteiro Arrow é passado diretamente.
- **Rust puro**: usa a crate `plotters` compilada sem dependências externas de sistema (*freetype*, *fontconfig* não são necessários), garantindo portabilidade em Linux, macOS e Windows.
- **Saída SVG**: gera SVG vetorial escalável, adequado para publicação, relatórios em $\text{\LaTeX}$ e inclusão em documentos web.
- **Composicional**: cada função recebe e retorna um `Dict`, tornando qualquer pipeline de plot testável, inspecionável e modificável.

1.3 Instalação

Instale o plugin diretamente do GitHub usando a CLI do Hayashi:

```
hay install sheep-farm/hayplot
```

Listing 1.1: Instalação via CLI

O binário pré-compilado para a sua plataforma é baixado automaticamente da última *GitHub Release*. Para verificar se a instalação ocorreu com sucesso:

```
hay list
```

Listing 1.2: Verificação da instalação

Você deve ver `sheep-farm/hayplot (native so plugin)` na lista de pacotes instalados.

Para atualizar para a versão mais recente:

```
hay update sheep-farm/hayplot
```

Listing 1.3: Atualização

Para verificar a integridade do que está instalado:

```
hay check-plugin sheep-farm/hayplot
```

Listing 1.4: Verificação de integridade

Para remover o plugin:

```
hay remove sheep-farm/hayplot
```

Listing 1.5: Remoção

1.4 Importando o Plugin

Em qualquer script `.hay`, importe o plugin com um alias:

```
1 import("sheep-farm/hayplot", as=gg)
```

Listing 1.6: Importação com alias

A partir daí, todas as funções são acessadas com o prefixo `gg::`.

Capítulo 2

Referência das Funções

hayplot expõe oito funções públicas. Cada uma recebe e retorna um dicionário de especificação de plot (*plot spec*), tornando a composição via pipe natural e segura.

2.1 hayplot(df, aes) — Inicialização

Inicializa um novo dicionário de especificação de plot com os dados e o mapeamento estético.

Assinatura

```
hayplot(df: DataFrame, aes: Dict) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
df	DataFrame	O DataFrame contendo as colunas a serem plotadas.
aes	Dict	Mapeamento estético. Deve conter as chaves "x" e "y" com os nomes das colunas.

Retorno

Um Dict com a estrutura interna da especificação de plot:

```
{
  "data":    <Arrow DataFrame>,
  "mapping": {"x": "coluna_x", "y": "coluna_y"},
  "layers":  [],
  "labs":    {}
}
```

Exemplo

```

1 let df = dataframe({"pib": [10, 20, 30], "vida": [70, 75, 80]})
2
3 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})

```

Listing 2.1: Criando a especificação inicial

2.2 geom_point(plot, color, size) — Camada de Pontos

Adiciona uma camada de dispersão (*scatter plot*) à especificação. Cada observação do DataFrame será representada por um círculo.

Assinatura

```
geom_point(plot: Dict, color: String, size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por hayplot().
color	String	Cor dos pontos. Valores aceitos: "red", "blue", "green", "yellow", "magenta", "cyan", "black". Padrão: "blue".
size	Float	Raio dos pontos em pixels. Valores recomendados: 4.0–12.0.

Retorno

O mesmo Dict da especificação com a camada de pontos adicionada ao campo "layers".

Exemplo

```

1 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})
2   |> gg::geom_point("red", 8.0)

```

Listing 2.2: Adicionando uma camada de dispersão

Atenção: Nota sobre cores

O nome da cor deve ser passado em inglês e minúsculas. Qualquer cor não reconhecida resultará na cor padrão "blue". O suporte a cores hexadecimais (#RRGGBB) está previsto para versões futuras.

2.3 geom_line(plot, color, size) — Camada de Linha

Adiciona uma camada de linha à especificação, conectando as observações do DataFrame em ordem. Ideal para séries temporais, tendências e gráficos de evolução. Pode ser combinado com `geom_point` para criar gráficos de linhas com marcadores.

Assinatura

```
geom_line(plot: Dict, color: String, size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 1.0–5.0.

Retorno

O mesmo Dict da especificação com a camada de linha adicionada ao campo "layers".

Composição de Camadas

As camadas são renderizadas na **ordem em que são adicionadas**. Para cobrir os pontos sobre a linha (evitando que a linha passe por cima), adicione `geom_line` *antes* de `geom_point`:

```
1 let plot = gg::hayplot(df, {"x": "mes", "y": "vendas"})
2   |> gg::geom_line("blue", 3.0)      // linha primeiro
3   |> gg::geom_point("red", 6.0)     // pontos por cima
4   |> gg::labs("Vendas Mensais", "Mes", "Vendas")
```

Listing 2.3: Gráfico de linha com marcadores de pontos

2.4 geom_bar(plot, color, width) — Camada de Barras

Adiciona uma camada de gráfico de barras à especificação. Cada observação do DataFrame será representada por uma barra vertical posicionada no valor *x* com altura correspondente ao valor *y*.

Assinatura

```
geom_bar(plot: Dict, color: String, width: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor das barras. Mesmos valores aceitos por <code>geom_point</code> .
width	Float	Largura das barras centradas no valor x . Valores recomendados: 0.4–1.0.

Retorno

O mesmo Dict da especificação com a camada de barras adicionada ao campo "layers".

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "categoria", "y": "vendas"})
2   |> gg::geom_bar("blue", 0.6)
3   |> gg::labs("Vendas por Categoria", "Categoria", "
   Vendas")
```

Listing 2.4: Gráfico de barras categórico

2.5 `geom_histogram(plot, color, bins)` — Camada de Histograma

Adiciona uma camada de histograma à especificação. Calcula automaticamente a distribuição de frequência dos valores da coluna y e divide em um número especificado de *bins* (intervalos).

Assinatura

```
geom_histogram(plot: Dict, color: String, bins: Int) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor das barras do histograma. Mesmos valores aceitos por <code>geom_point</code> .
bins	Int	Número de intervalos (<i>bins</i>) para dividir os dados. Valores recomendados: 5–30.

Retorno

O mesmo Dict da especificação com a camada de histograma adicionada ao campo "layers".

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "dummy", "y": "notas"})
2       |> gg::geom_histogram("green", 15)
3       |> gg::labs("Distribuição de Notas", "Nota", "
      Frequência")

```

Listing 2.5: Histograma de distribuição

Dica: Dummy x para histogramas

Para histogramas univariados, use uma coluna x constante (e.g., "dummy": [1.0, 1.0, ...]) pois o histograma calcula a distribuição apenas a partir dos valores y .

2.6 `geom_boxplot(plot, color, width)` — Camada de Boxplot

Adiciona uma camada de boxplot (*box-and-whisker plot*) à especificação. Exibe estatísticas resumidas: quartis (Q1, Q3), mediana, e *whiskers* ($1.5 \times \text{IQR}$). Ideal para visualizar distribuição e identificar outliers.

Assinatura

```
geom_boxplot(plot: Dict, color: String, width: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor do box e whiskers. Mesmos valores aceitos por <code>geom_point</code> .
width	Float	Largura da caixa. Valores recomendados: 0.3–0.7.

Retorno

O mesmo Dict da especificação com a camada de boxplot adicionada ao campo "layers".

Estatísticas Calculadas

- **Q1:** 25º percentil (limite inferior da caixa)
- **Mediana:** 50º percentil (linha preta na caixa)
- **Q3:** 75º percentil (limite superior da caixa)
- **IQR:** $Q3 - Q1$ (intervalo interquartil)
- **Whiskers:** $Q1 - 1.5 \times IQR$ e $Q3 + 1.5 \times IQR$ (limites dos whiskers)

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "departamento", "y": "salario"
2     })
3     |> gg::geom_boxplot("red", 0.4)
4     |> gg::labs("Distribuição Salarial", "Departamento",
5     "Salário (USD)")

```

Listing 2.6: Boxplot de distribuição salarial

2.7 `geom_heatmap(plot, color, cell_size)` — Camada de Heatmap

Adiciona uma camada de heatmap (*mapa de calor*) à especificação. Cada par (x, y) é renderizado como uma célula retangular cuja intensidade de cor é proporcional ao valor y normalizado. Ideal para visualizar dados em grade 2D.

Assinatura

```
geom_heatmap(plot: Dict, color: String, cell_size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor base para o heatmap. A intensidade é calculada misturando com branco.
cell_size	Float	Tamanho das células do heatmap. Valores recomendados: 0.5–1.5.

Retorno

O mesmo Dict da especificação com a camada de heatmap adicionada ao campo "layers".

Normalização de Cores

Os valores y são normalizados para o intervalo $[0, 1]$ usando a fórmula:

$$intensidade = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$$

A cor final é calculada misturando a cor base com branco proporcionalmente à intensidade (maior intensidade = cor mais saturada).

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "coord_x", "y": "coord_y"})
2   |> gg::geom_heatmap("red", 0.8)
3   |> gg::labs("Temperatura", "Coordenada X", "
    Coordenada Y")

```

Listing 2.7: Heatmap de temperatura em grade

2.8 `geom_area(plot, color, size)` — Camada de Área

Adiciona uma camada de gráfico de área à especificação. Preenche a área sob a linha conectando os pontos, ordenando automaticamente por x . Ideal para visualizar valores acumulados, volume ao longo do tempo ou PnL.

Assinatura

```
geom_area(plot: Dict, color: String, size: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da área e da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha de contorno. Valores recomendados: 1.0–3.0.

Retorno

O mesmo Dict da especificação com a camada de área adicionada ao campo "layers".

Comportamento

Os pontos são automaticamente ordenados por x antes do rendering. A área é preenchida usando uma aproximação por polígonos entre pontos consecutivos, com base em $y = 0$ como limite inferior.

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "mes", "y": "receita_acumulada"
2   })
3   |> gg::geom_area("blue", 2.0)
4   |> gg::labs("Receita Acumulada", "Mês", "Receita (
5     milhares)")

```

Listing 2.8: Gráfico de área para receita acumulada

Dica: Ordenação automática

Diferente de `geom_line`, `geom_area` ordena automaticamente os pontos por x antes de renderizar, garantindo que a área seja preenchida corretamente mesmo se os dados não estiverem ordenados.

2.9 `geom_hline(plot, color, size, yintercept)` — Linha Horizontal

Adiciona uma linha horizontal de referência ao gráfico em um valor y específico. Ideal para marcar thresholds, limites de risco, médias ou valores alvo.

Assinatura

```

geom_hline(plot: Dict, color: String, size: Float, yintercept:
  Float) -> Dict

```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 0.5–2.0.
yintercept	Float	Valor y onde a linha horizontal será desenhada.

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "tempo", "y": "valor"})
2   |> gg::geom_line("blue", 2.0)
3   |> gg::geom_hline("red", 1.0, 50.0) // Linha em y=50
4   |> gg::labs("Métrica com Threshold", "Tempo", "Valor"
5             )

```

Listing 2.9: Linha horizontal de referência

2.10 `geom_vline(plot, color, size, xintercept)` — Linha Vertical

Adiciona uma linha vertical de referência ao gráfico em um valor x específico. Ideal para marcar eventos, split de ações, marcos temporais ou pontos de corte.

Assinatura

```
geom_vline(plot: Dict, color: String, size: Float, xintercept:
  Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 0.5–2.0.
xintercept	Float	Valor x onde a linha vertical será desenhada.

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "ano", "y": "receita"})
2   |> gg::geom_vline("blue", 2.0)

```

```

3 |> gg::geom_vline("green", 1.0, 2020.0) // Linha em
   x=2020
4 |> gg::labs("Receita com Evento Marcado", "Ano", "
   Receita")

```

Listing 2.10: Linha vertical de evento

2.11 `geom_abline(plot, color, size, slope, intercept)` — Linha Diagonal

Adiciona uma linha diagonal de referência seguindo a equação $y = slope \times x + intercept$. Ideal para linhas de 45° (benchmark), tendências teóricas ou relações lineares esperadas.

Assinatura

```
geom_abline(plot: Dict, color: String, size: Float, slope: Float,
  intercept: Float) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 0.5–2.0.
slope	Float	Coefficiente angular da linha.
intercept	Float	Intercepto da linha (valor em y quando $x = 0$).

Exemplo

2.12 `geom_step(plot, color, size, direction)` — Li- nha em Degrau

Adiciona uma linha em degrau conectando os pontos. Útil para funções de distribuição acumulada, preços discretos ou dados que mudam em passos. A direção controla se o degrau é horizontal-então-vertical ("hv") ou vertical-então-horizontal ("vh").

Assinatura

```
geom_step(plot: Dict, color: String, size: Float, direction: String
) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .
color	String	Cor da linha. Mesmos valores aceitos por <code>geom_point</code> .
size	Float	Espessura da linha em pixels. Valores recomendados: 1.0–3.0.
direction	String	Direção do degrau: "hv" (horizontal-então-vertical) ou "vh" (vertical-então-horizontal).

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "tempo", "y": "preco"})
2   |> gg::geom_step("blue", 2.0, "hv")
3   |> gg::labs("Mudanças de Preço (Degrau)", "Tempo", "
    Preço")

```

Listing 2.11: Gráfico de degrau para preços

2.13 `scale_x_log10(plot)` — Escala Logarítmica em X

Define o eixo x para escala logarítmica base 10. Ideal para dados com crescimento exponencial ou ranges muito amplos. Valores não-positivos (≤ 0) são tratados como NaN.

Assinatura

```
scale_x_log10(plot: Dict) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "frequencia", "y": "amplitude"
2   })
3   |> gg::geom_point("blue", 5.0)
4   |> gg::scale_x_log10()
5   |> gg::labs("Resposta em Frequência", "Frequência (
    log10)", "Amplitude")

```


Listing 2.12: Eixo X em escala logarítmica

2.14 `scale_y_log10(plot)` — Escala Logarítmica em Y

Define o eixo *y* para escala logarítmica base 10. Ideal para dados com crescimento exponencial ou ranges muito amplos. Valores não-positivos (≤ 0) são tratados como NaN.

Assinatura

```
scale_y_log10(plot: Dict) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot retornada por <code>hayplot()</code> .

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "tempo", "y": "valor"})
2   |> gg::geom_line("red", 2.0)
3   |> gg::scale_y_log10()
4   |> gg::labs("Crescimento Exponencial", "Tempo", "
   Valor (log10)")
```

Listing 2.13: Eixo Y em escala logarítmica

Dica: Cores hexadecimais

Todos os parâmetros de cor aceitam tanto cores nomeadas ("red", "blue", etc.) quanto códigos hexadecimais no formato #RRGGBB (ex: "#FF5733", "#C70039"). Isso permite especificar cores personalizadas com precisão.

2.15 `filter_data(df, col, value)` — Filtragem de Dados

Filtra um DataFrame para incluir apenas as linhas onde a coluna especificada é igual ao valor dado. Esta é uma abordagem simplificada de faceting — filtre os dados manualmente, depois chame `hayplot` para cada grupo.

Assinatura

```
filter_data(df: DataFrame, col: String, value: Float) -> Result<
  DataFrame, String>
```

Parâmetros

Parâmetro	Tipo	Descrição
df	DataFrame	O DataFrame a ser filtrado.
col	String	Nome da coluna numérica usada para filtragem.
value	Float	Valor para igualar (tolerância de 1e-9).

Retorno

Um novo DataFrame contendo apenas as linhas onde `col == value`.

Exemplo

```
1 let df1 = gg::filter_data(df, "categoria", 1.0)
2 let plot1 = gg::hayplot(df1, {"x": "tempo", "y": "valor"})
3           |> gg::geom_point("blue", 5.0)
4           |> gg::labs("Categoria 1", "Tempo", "Valor")
5 let svg1 = gg::render_svg(plot1)
```

Listing 2.14: Faceting manual com filter_data

Nota: Faceting manual

Para faceting, filtre os dados usando `filter_data()` para cada grupo e chame `hayplot` manualmente. As funções `facet_wrap` e `render_facets` foram descontinuadas em favor desta abordagem.

2.16 set_dimensions(plot, width, height) — Dimensões do SVG

Define as dimensões de saída do SVG em pixels. O padrão é 800x600.

Assinatura

```
set_dimensions(plot: Dict, width: Int, height: Int) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
width	Int	Largura do SVG em pixels (padrão: 800).
height	Int	Altura do SVG em pixels (padrão: 600).

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2       |> gg::set_dimensions(1200, 800)
3       |> gg::geom_point("blue", 5.0)

```

Listing 2.15: Dimensões customizadas

2.17 set_margins(plot, top, bottom, left, right) — Margens do Plot

Define as margens do plot em pixels. O padrão é 20px em todos os lados.

Assinatura

```

set_margins(plot: Dict, top: Int, bottom: Int, left: Int, right:
  Int) -> Dict

```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
top	Int	Margem superior em pixels (padrão: 20).
bottom	Int	Margem inferior em pixels (padrão: 20).
left	Int	Margem esquerda em pixels (padrão: 20).
right	Int	Margem direita em pixels (padrão: 20).

Exemplo

```

1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2       |> gg::set_margins(40, 40, 60, 40)
3       |> gg::geom_point("blue", 5.0)

```

Listing 2.16: Margens customizadas

2.18 save_svg(plot, filename) — Salvar SVG

Renderiza o plot e salva diretamente em um arquivo. Função de conveniência que combina `render_svg()` + `write()`.

Assinatura

```
save_svg(plot: Dict, filename: String) -> Result<String, String>
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
filename	String	Nome do arquivo para salvar o SVG.

Retorno

O conteúdo SVG como uma string, caso seja necessário para processamento adicional.

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2     |> gg::geom_point("blue", 5.0)
3     |> gg::geom_line("red", 2.0)
4 let svg = gg::save_svg(plot, "output.svg")
```

Listing 2.17: Salvamento em um passo

2.19 set_background_color(plot, color) — Cor de Fundo

Define a cor de fundo do plot. O padrão é branco. Aceita cores nomeadas ou códigos hexadecimais.

Assinatura

```
set_background_color(plot: Dict, color: String) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
color	String	Nome da cor ou código hex (ex: "white", "#000000").

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2       |> gg::set_background_color("black")
3       |> gg::geom_point("white", 5.0)
```

Listing 2.18: Tema escuro

2.20 set_grid(plot, show_grid) — Controle do Grid

Habilita ou desabilita o grid do plot. O padrão é `true` (grid habilitado).

Assinatura

```
set_grid(plot: Dict, show_grid: Bool) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
show_grid	Bool	<code>true</code> para habilitar, <code>false</code> para desabilitar.

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2       |> gg::set_grid(false)
3       |> gg::geom_point("blue", 5.0)
```

Listing 2.19: Plot sem grid

2.21 save_png(plot, filename) — Salvar PNG

Renderiza o plot e salva como arquivo PNG. Requer a feature `png` durante a compilação. Função de conveniência que combina renderização PNG + escrita de arquivo.

Assinatura

```
save_png(plot: Dict, filename: String) -> Result<String, String>
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
filename	String	Nome do arquivo para salvar o PNG.

Retorno

O conteúdo PNG codificado em base64 como uma string.

Nota

Requer compilação com a feature `png`: `cargo build --release --features png`. O backend PNG atualmente suporta apenas geometrias básicas (point, line, bar, area).

Exemplo

```
1 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
2     |> gg::geom_point("blue", 5.0)
3     |> gg::geom_line("red", 2.0)
4 let png = gg::save_png(plot, "output.png")
```

Listing 2.20: Salvamento PNG em um passo

2.22 labs(plot, title, x, y) — Rótulos

Define o título do gráfico e os rótulos dos eixos horizontal e vertical.

Assinatura

```
labs(plot: Dict, title: String, x: String, y: String) -> Dict
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot.
title	String	Título exibido no topo do gráfico.
x	String	Rótulo do eixo horizontal.
y	String	Rótulo do eixo vertical.

Retorno

O mesmo Dict com o campo "labs" preenchido.

Exemplo

```

1 let spec = gg::hayplot(df, {"x": "pib", "y": "vida"})
2   |> gg::geom_point("blue", 6.0)
3   |> gg::labs("PIB vs. Expectativa de Vida",
4               "PIB per Capita (USD)",
5               "Expectativa de Vida (anos)")

```

Listing 2.21: Definindo título e nomes dos eixos

2.23 render_svg(plot) — Renderização

Compila a especificação de plot e renderiza o gráfico final como uma **string SVG**. Esta é a única função que acessa os dados do DataFrame e executa operações de desenho.

Assinatura

```
render_svg(plot: Dict) -> Result<String, String>
```

Parâmetros

Parâmetro	Tipo	Descrição
plot	Dict	A especificação de plot completa (com dados, mapeamento, camadas e rótulos).

Retorno

- Em caso de sucesso: uma **String** contendo o código XML do SVG (800×600 pixels).
- Em caso de erro: uma mensagem de erro descritiva como **String**.

Nota: Dimensões do SVG

A saída SVG tem resolução fixa de **800×600 pixels**. Como é um formato vetorial, pode ser redimensionada sem perda de qualidade em qualquer aplicação ou editor de imagens.

Processo interno de renderização

Internamente, `render_svg()` executa os seguintes passos:

1. Extrai o ponteiro Arrow do campo "data" via FFI Zero-Copy;
2. Lê o mapeamento "x" e "y" e recupera as colunas do DataFrame;
3. Calcula os limites automáticos dos eixos com 10% de margem;

4. Itera sobre "layers" e aplica cada geometria;
5. Compila o gráfico para uma string SVG em memória usando `plotters`.

Exemplo

```
1 let svg_content = gg::render_svg(plot)
2 write(svg_content, "saida.svg")
```

Listing 2.22: Renderizando e salvando o SVG

Capítulo 3

Fluxo de Trabalho

3.1 Pipeline Declarativo

O fluxo típico de um gráfico em **hayplot** segue a estrutura abaixo:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar ou criar os dados
4 load "dados.csv" as df
5
6 // 2. Inicializar a especificacao com dados e mapeamento
7 // 3. Adicionar camadas geometricas
8 // 4. Definir rotulos
9 let plot = gg::hayplot(df, {"x": "col_x", "y": "col_y"})
10      |> gg::geom_point("blue", 6.0)
11      |> gg::labs("Titulo do Grafico", "Eixo X", "Eixo Y")
12
13 // 5. Renderizar e salvar
14 let svg = gg::render_svg(plot)
15 write(svg, "grafico.svg")
```

Listing 3.1: Estrutura geral de um pipeline hayplot

3.2 Tipos de Colunas Suportados

A função `render_svg()` aceita colunas numéricas nos seguintes tipos Arrow:

Tipo Arrow	Tipo Hayashi
Float64	float
Int64	int

Atenção: Colunas categóricas

Colunas do tipo **String** ou booleanas **não** são suportadas como coordenadas x ou y . Para variáveis categóricas, encode-as numericamente antes de plotar.

3.3 Valores Ausentes

Observações com NaN em qualquer uma das coordenadas são automaticamente omitidas da renderização. Não é necessário filtrar os dados antes de plotar.

Capítulo 4

Exemplos Completos

4.1 Dispersão Simples

O exemplo mais básico: dados criados inline, sem arquivos externos.

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Criar dataset PIB vs Expectativa de Vida
4 let data = {
5     "gdp_per_capita": [12000, 24000, 35000, 48000, 65000,
6                       85000],
7     "life_expectancy": [68.5, 72.1, 76.4, 79.2, 81.5, 83.1]
8 }
9
10 let df = dataframe(data)
11
12 // 2. Construir a especificacao do plot
13 print("Building basic scatter plot...")
14 let plot = gg::hayplot(df, {"x": "gdp_per_capita", "y": "
15     life_expectancy"})
16     |> gg::geom_point("blue", 7.0)
17     |> gg::labs(
18         "GDP vs Life Expectancy",
19         "GDP per Capita (USD)",
20         "Life Expectancy (Years)"
21     )
22
23 // 3. Renderizar e salvar
24 let svg = gg::render_svg(plot)
25 write(svg, "gdp_life_exp.svg")
26 print("Plot saved to 'gdp_life_exp.svg'!")
```

Listing 4.1: examples/basic_scatter.hay

4.2 Curva de Phillips

Clássico da macroeconomia: relação negativa entre desemprego e inflação.

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Dataset macroeconomico simulado
4 let data = {
5   "unemployment_rate": [3.5, 4.2, 5.0, 6.1, 7.5, 8.8, 9.5],
6   "inflation_rate":    [5.2, 4.5, 3.8, 2.9, 1.8, 1.2, 0.9]
7 }
8 let df = dataframe(data)
9
10 // 2. Construir o grafico da Curva de Phillips
11 print("Building Phillips Curve plot...")
12 let plot = gg::hayplot(df, {"x": "unemployment_rate", "y": "
    inflation_rate"})
13     |> gg::geom_point("red", 8.0)
14     |> gg::labs(
15       "The Phillips Curve",
16       "Unemployment Rate (%)",
17       "Inflation Rate (%)")
18
19
20 // 3. Renderizar e salvar
21 let svg = gg::render_svg(plot)
22 write(svg, "phillips_curve.svg")
23 print("Plot saved to 'phillips_curve.svg'!")
```

Listing 4.2: examples/phillips_curve.hay

4.3 A partir de Arquivo CSV

Integrando **hayplot** com o comando **load** do Hayashi:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar dados externos
4 load "dados_painel.csv" as df
5
6 // 2. Criar grafico
7 let plot = gg::hayplot(df, {"x": "renda", "y": "consumo"})
8     |> gg::geom_point("green", 5.0)
9     |> gg::labs(
10       "Renda vs. Consumo",
11       "Renda (R$)",
12       "Consumo (R$)")
13
14
15 // 3. Renderizar
16 let svg = gg::render_svg(plot)
17 write(svg, "renda_consumo.svg")
```

Listing 4.3: Plot a partir de um arquivo externo

4.4 Combinando com Regressão OLS

hayplot integra-se naturalmente ao workflow econométrico do Hayashi. O exemplo a seguir estima um modelo OLS e visualiza os valores ajustados contra os observados:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // 1. Carregar dados e estimar OLS
4 load "painel.csv" as df
5 let m = ols(Y ~ X1 + X2, df)
6
7 // 2. Gerar valores preditos
8 predict df yhat = m
9
10 // 3. Scatter: observado vs predito
11 let plot = gg::hayplot(df, {"x": "yhat", "y": "Y"})
12     |> gg::geom_point("blue", 5.0)
13     |> gg::labs(
14         "Observado vs. Ajustado",
15         "Y-hat (valores preditos)",
16         "Y (valores observados)"
17     )
18
19 // 4. Salvar
20 let svg = gg::render_svg(plot)
21 write(svg, "obs_vs_fit.svg")
```

Listing 4.4: Visualizando valores ajustados de uma regressão OLS

4.5 Pipeline Multi-linha

Com o suporte a quebra de linha no operador `|>` do Hayashi, é possível escrever pipelines mais legíveis:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 let df = dataframe({"x": [1.0, 2.0, 3.0, 4.0, 5.0],
4                     "y": [2.5, 4.1, 5.9, 8.2, 10.0]})
5
6 let svg = gg::hayplot(df, {"x": "x", "y": "y"})
7     |> gg::geom_point("cyan", 9.0)
8     |> gg::labs("Tendencia Linear", "Variavel X", "Variavel Y")
9     |> gg::render_svg
10
11 write(svg, "tendencia.svg")
```

Listing 4.5: Usando quebras de linha na pipeline

4.6 Linha com Marcadores de Pontos

Combinando `geom_line` e `geom_point` em uma única pipeline, é possível criar gráficos de tendência com marcadores visuais em cada observação. A **ordem das camadas** determina qual é desenhada por cima:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de crescimento de vendas mensais
4 let data = {"month": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
5                 9.0, 10.0, 11.0, 12.0],
6             "sales": [10.5, 12.0, 11.2, 14.8, 16.5, 15.0, 18.2,
7                     21.0, 19.5, 22.8, 25.0, 24.2]}
8 let df = dataframe(data)
9
10 // geom_line adicionado antes de geom_point para que os pontos
11 // fiquem por cima
12 let plot = gg::hayplot(df, {"x": "month", "y": "sales"})
13   |> gg::geom_line("blue", 3.0)
14   |> gg::geom_point("red", 6.0)
15   |> gg::labs("Sales Growth Over Time", "Month", "Sales (
16             Thousands)")
17
18 let svg_content = gg::render_svg(plot)
19 write(svg_content, "sales_growth.svg")
20 print("Plot saved to 'sales_growth.svg'!")
```

Listing 4.6: examples/line_and_scatter.hay

4.7 Gráfico de Barras

Visualização de dados categóricos ou agregados por grupos:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de vendas por categoria
4 let data = {"category": [1.0, 2.0, 3.0, 4.0, 5.0],
5             "sales": [150.0, 230.0, 180.0, 320.0, 290.0]}
6 let df = dataframe(data)
7
8 let plot = gg::hayplot(df, {"x": "category", "y": "sales"})
9   |> gg::geom_bar("blue", 0.6)
10   |> gg::labs("Sales by Category", "Category", "Sales (
11             Thousands)")
12
13 let svg_content = gg::render_svg(plot)
14 write(svg_content, "bar_chart.svg")
15 print("Plot saved to 'bar_chart.svg'!")
```

Listing 4.7: examples/bar_chart.hay

4.8 Histograma

Análise de distribuição de uma variável univariada:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de notas de teste
4 let data = {"dummy": [1.0, 1.0, 1.0, 1.0, 1.0],
5             "scores": [65.0, 72.0, 78.0, 85.0, 88.0, 90.0, 92.0,
6                       95.0,
7                       78.0, 82.0, 75.0, 88.0, 91.0, 84.0, 79.0,
8                       86.0,
9                       93.0, 77.0, 83.0, 89.0]}
10 let df = dataframe(data)
11
12 let plot = gg::hayplot(df, {"x": "dummy", "y": "scores"})
13 |> gg::geom_histogram("green", 15)
14 |> gg::labs("Distribution of Test Scores", "Score", "
15           Frequency")
16
17 let svg_content = gg::render_svg(plot)
18 write(svg_content, "histogram.svg")
19 print("Plot saved to 'histogram.svg'!")
```

Listing 4.8: examples/histogram.hay

4.9 Boxplot

Visualização de distribuição e identificação de outliers:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados salariais por departamento
4 let data = {"department": [1.0, 1.0, 1.0, 1.0, 1.0],
5             "salary": [45000.0, 52000.0, 48000.0, 61000.0,
6                       55000.0,
7                       58000.0, 49000.0, 63000.0, 51000.0,
8                       57000.0]}
9 let df = dataframe(data)
10
11 let plot = gg::hayplot(df, {"x": "department", "y": "salary"})
12 |> gg::geom_boxplot("red", 0.4)
13 |> gg::labs("Salary Distribution", "Department", "Salary (
14           USD)")
15
16 let svg_content = gg::render_svg(plot)
17 write(svg_content, "boxplot.svg")
18 print("Plot saved to 'boxplot.svg'!")
```

Listing 4.9: examples/boxplot.hay

4.10 Heatmap

Visualização de dados em grade 2D com intensidade de cor:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de temperatura em grade 3x3
4 let data = {"x": [1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 1.0, 2.0, 3.0],
5             "y": [1.0, 1.0, 1.0, 2.0, 2.0, 2.0, 3.0, 3.0, 3.0],
6             "temperature": [20.0, 25.0, 30.0, 22.0, 28.0, 35.0,
7                             18.0, 24.0, 32.0]}
8 let df = dataframe(data)
9
10 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
11     |> gg::geom_heatmap("red", 0.8)
12     |> gg::labs("Temperature Heatmap", "X Location", "Y Location")
13
14 let svg_content = gg::render_svg(plot)
15 write(svg_content, "heatmap.svg")
16 print("Plot saved to 'heatmap.svg'!")
```

Listing 4.10: examples/heatmap.hay

4.11 Gráfico de Área

Visualização de valores acumulados com área preenchida:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de receita acumulada mensal
4 let data = {"month": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
5                      9.0, 10.0, 11.0, 12.0],
6             "revenue": [10.0, 25.0, 45.0, 70.0, 100.0, 135.0,
7                          175.0, 220.0,
8                          270.0, 325.0, 385.0, 450.0]}
9 let df = dataframe(data)
10
11 let plot = gg::hayplot(df, {"x": "month", "y": "revenue"})
12     |> gg::geom_area("blue", 2.0)
13     |> gg::labs("Cumulative Revenue Over Time", "Month", "Revenue (Thousands)")
14
15 let svg_content = gg::render_svg(plot)
16 write(svg_content, "area_chart.svg")
17 print("Plot saved to 'area_chart.svg'!")
```

Listing 4.11: examples/area_chart.hay

4.12 Linhas de Referência

Combinação de linhas horizontais, verticais e diagonais para anotações:

```

1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de vendas mensais
4 let data = {"month": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
5                   9.0, 10.0, 11.0, 12.0],
6             "sales": [10.5, 12.0, 11.2, 14.8, 16.5, 15.0, 18.2,
7                   21.0,
8                   19.5, 22.8, 25.0, 24.2]}
9 let df = dataframe(data)
10
11 let plot = gg::hayplot(df, {"x": "month", "y": "sales"})
12 |> gg::geom_line("blue", 2.0)
13 |> gg::geom_hline("red", 1.0, 15.0) // Horizontal line at
14 |> gg::geom_vline("green", 1.0, 6.0) // Vertical line at x
15 |> gg::geom_abline("magenta", 1.0, 1.5, 8.0) // Diagonal
16 |> gg::labs("Sales with Reference Lines", "Month", "Sales (
17   Thousands)")
18
19 let svg_content = gg::render_svg(plot)
20 write(svg_content, "reference_lines.svg")
21 print("Plot saved to 'reference_lines.svg'!")

```

Listing 4.12: examples/reference_lines.hay

4.13 Gráfico de Degrau

Visualização de mudanças discretas com linhas em degrau:

```

1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados de preços em tempo discreto
4 let data = {"time": [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0],
5             "price": [100.0, 105.0, 103.0, 108.0, 106.0, 110.0,
6                   109.0, 112.0]}
7 let df = dataframe(data)
8
9 let plot = gg::hayplot(df, {"x": "time", "y": "price"})
10 |> gg::geom_step("blue", 2.0, "hv") // horizontal then
11 |> gg::labs("Price Changes (Step Chart)", "Time", "Price")
12
13 let svg_content = gg::render_svg(plot)
14 write(svg_content, "step_chart.svg")
15 print("Plot saved to 'step_chart.svg'!")

```

Listing 4.13: examples/step_chart.hay

4.14 Escala Logarítmica

Visualização de dados exponenciais com escala logarítmica e cores hexadecimais:

```

1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados com crescimento exponencial
4 let data = {"x": [1.0, 10.0, 100.0, 1000.0, 10000.0, 100000.0],
5             "y": [1.0, 100.0, 10000.0, 1000000.0, 100000000.0,
6                 10000000000.0]}
7
8 let df = dataframe(data)
9
10 let plot = gg::hayplot(df, {"x": "x", "y": "y"})
11 |> gg::geom_point("#FF5733", 5.0) // Custom hex color
12 |> gg::geom_line("#C70039", 2.0)
13 |> gg::scale_x_log10()
14 |> gg::scale_y_log10()
15 |> gg::labs("Exponential Growth (Log Scale)", "X (log10)", "
16             Y (log10)")
17
18 let svg_content = gg::render_svg(plot)
19 write(svg_content, "log_scale.svg")
20 print("Plot saved to 'log_scale.svg'!")

```

Listing 4.14: examples/log_scale.hay

4.15 Faceting Manual

Visualização de dados agrupados usando filtragem manual de dados:

```

1 import("sheep-farm/hayplot", as=gg)
2
3 // Dados com grupos
4 let data = {"x": [1.0, 2.0, 3.0, 1.0, 2.0, 3.0, 1.0, 2.0, 3.0],
5             "y": [10.0, 20.0, 30.0, 15.0, 25.0, 35.0, 12.0,
6                 22.0, 32.0],
7             "group": [1.0, 1.0, 1.0, 2.0, 2.0, 2.0, 3.0, 3.0,
8                     3.0]}
9
10 let df = dataframe(data)
11
12 // Filtrar e renderizar cada grupo manualmente
13 let df1 = gg::filter_data(df, "group", 1.0)
14 let plot1 = gg::hayplot(df1, {"x": "x", "y": "y"})
15 |> gg::geom_point("blue", 5.0)
16 |> gg::geom_line("red", 2.0)
17 |> gg::labs("Group 1", "X", "Y")

```

```
15 let svg1 = gg::render_svg(plot1)
16 write(svg1, "facet_group_1.svg")
17
18 let df2 = gg::filter_data(df, "group", 2.0)
19 let plot2 = gg::hayplot(df2, {"x": "x", "y": "y"})
20   |> gg::geom_point("green", 5.0)
21   |> gg::geom_line("orange", 2.0)
22   |> gg::labs("Group 2", "X", "Y")
23 let svg2 = gg::render_svg(plot2)
24 write(svg2, "facet_group_2.svg")
25
26 let df3 = gg::filter_data(df, "group", 3.0)
27 let plot3 = gg::hayplot(df3, {"x": "x", "y": "y"})
28   |> gg::geom_point("purple", 5.0)
29   |> gg::geom_line("magenta", 2.0)
30   |> gg::labs("Group 3", "X", "Y")
31 let svg3 = gg::render_svg(plot3)
32 write(svg3, "facet_group_3.svg")
33
34 print("All facet plots saved successfully!")
```

Listing 4.15: examples/filter_facet.hay

Capítulo 5

Arquitetura Técnica

5.1 Comunicação via FFI Arrow

hayplot é um **plugin nativo** do Hayashi: uma biblioteca dinâmica (`.so/.dll/.dylib`) carregada em tempo de execução. A comunicação entre o interpretador e o plugin segue o protocolo de FFI do **hayashi-plugin-sdk**:

1. O interpretador serializa os argumentos em uma estrutura C-ABI;
2. DataFrames são passados como ponteiros Arrow **ArrayRef**, sem cópia dos dados;
3. O plugin processa em Rust nativo e retorna os valores;
4. O interpretador desserializa o retorno e libera a memória via hooks de desalocação.

Nota: Por que Zero-Copy?

DataFrames econométricos podem ter milhões de linhas. Passar os dados via serialização (JSON, por exemplo) causaria duplicação de memória e *overhead* significativo de CPU. O Arrow FFI resolve isso com um protocolo de ponteiros C padronizado pela Apache Software Foundation.

5.2 Dependências

Crate	Versão	Função
hayashi-plugin-sdk	<i>main</i>	FFI ABI, tipos Arrow, macros de exportação
plotters	0.3.7	Renderização SVG, primitivas gráficas
arrow	(via SDK)	Tipos de dados colunares

plotters é compilado com `default-features = false` e apenas com as features `svg_backend`, `area_series` e `line_series`, eliminando todas as dependências de sistema.

5.3 Compilação e Distribuição

O plugin é compilado como `cdylib` (biblioteca C dinâmica):

```
[lib]
crate-type = ["cdylib"]
```

Listing 5.1: Cargo.toml — tipo de crate

O binário final é distribuído via **GitHub Releases** com CI/CD automático. O Hayashi baixa automaticamente o artefato correto para a arquitetura alvo:

Plataforma	Arquivo
Linux x86_64	hayplot-x86_64-unknown-linux-gnu.so
macOS aarch64	hayplot-aarch64-apple-darwin.dylib
Windows x86_64	hayplot-x86_64-pc-windows-msvc.dll

Capítulo 6

Guia de Contribuição

6.1 Configurando o Ambiente

```
git clone https://github.com/sheep-farm/hayplot.git
cd hayplot
cargo build
```

Listing 6.1: Clonar e compilar localmente

Para compilar em modo de lançamento (otimizado):

```
cargo build --release
```

O arquivo `.so` estará em `target/release/libhayplot.so`.

6.2 Testando com o Hayashi Local

Para usar a versão local do plugin (sem instalar pelo CLI):

```
1 import("./target/debug/libhayplot.so", as=gg)
```

Listing 6.2: Carregando o `.so` compilado localmente

Atenção

O caminho com `./target/debug/` é adequado apenas para desenvolvimento. Em produção, use sempre `import("sheep-farm/hayplot", as=gg)` para carregar a versão instalada via `hay install`.

6.3 Adicionando Novas Geometrias

O padrão para adicionar uma nova geometria (e.g., `geom_line`) é:

1. Criar uma função pública anotada com `#[hayashi_fn]`;
2. A função recebe `mut plot: HashMap<String, HayashiValue>`;
3. Adiciona uma entrada ao vetor `"layers"` com a chave `"geom"`;

4. Retorna o plot modificado.

```
1 #[hayashi_fn]
2 pub fn geom_line(
3     mut plot: HashMap<String, HayashiValue>,
4     color: String,
5     width: f64,
6 ) -> HashMap<String, HayashiValue> {
7     if let Some(HayashiValue::List(ref mut layers)) = plot.
8         get_mut("layers") {
9         let mut layer = HashMap::new();
10        layer.insert("geom".to_string(), HayashiValue::Str("
11            line".to_string()));
12        layer.insert("color".to_string(), HayashiValue::Str(
13            color));
14        layer.insert("width".to_string(), HayashiValue::Float(
15            width));
16        layers.push(HayashiValue::Dict(layer));
17    }
18    plot
19 }
```

Listing 6.3: Estrutura de uma nova geometria em Rust

Em seguida, adicione o tratamento correspondente dentro de `render_svg()` no loop de layers.

Capítulo 7

Roadmap

Versão	Feature	Status
v0.1.x	<code>geom_point</code> — dispersão	✓ Concluído
v0.1.x	<code>geom_line</code> — séries temporais e tendência	✓ Concluído
v0.3.0	<code>geom_bar</code> — barras verticais	✓ Concluído
v0.3.0	<code>geom_histogram</code> — histograma	✓ Concluído
v0.3.0	<code>geom_boxplot</code> — box-and-whisker	✓ Concluído
v0.3.0	<code>geom_heatmap</code> — mapa de calor 2D	✓ Concluído
v0.4.0	<code>geom_area</code> — área preenchida sob linha	✓ Concluído
v0.5.0	<code>geom_hline</code> — linha horizontal	✓ Concluído
v0.5.0	<code>geom_vline</code> — linha vertical	✓ Concluído
v0.5.0	<code>geom_abline</code> — linha diagonal	✓ Concluído
v0.5.0	<code>geom_step</code> — linhas em degrau	✓ Concluído
v0.6.0	Suporte a cores hexadecimais (<code>#RRGGBB</code>)	✓ Concluído
v0.6.0	<code>scale_x_log10</code> , <code>scale_y_log10</code> — eixos em log	✓ Concluído
v0.8.0	<code>filter_data</code> — filtragem de DataFrames	✓ Concluído
v0.8.0	<code>facet_wrap</code> , <code>render_facets</code> — descontinuos	✓ Substituídos por <code>filter_data</code>
v0.9.0	<code>set_dimensions</code> — tamanho customizável de SVG	✓ Concluído
v0.9.0	<code>set_margins</code> — margens customizáveis	✓ Concluído
v0.9.0	<code>save_svg</code> — render + save em um passo	✓ Concluído
v1.0.0	<code>set_background_color</code> — cor de fundo customizável	✓ Concluído
v1.0.0	<code>set_grid</code> — controle do grid on/off	✓ Concluído
v1.1.0	<code>save_png</code> — exportação PNG (feature flag)	✓ Concluído
v1.1.0	Suporte completo de geometrias em PNG	Futuro
v1.1.0	<code>geom_ribbon</code> — bandas de intervalo de confiança	Futuro
v1.1.0	<code>geom_smooth</code> — regressão LOESS/linear	Futuro
v1.1.0	<code>theme_*</code> — temas de estilo (dark, minimal, classic)	Futuro

Capítulo 8

Referência Rápida

Função	Entrada	Descrição
<code>hayplot(df, aes)</code>	DataFrame, Dict	Inicializa a spec com dados e mapeamento x/y
<code>geom_point(plot, color, size)</code>	Dict, String, Float	Adiciona camada de dispersão (círculos)
<code>geom_line(plot, color, size)</code>	Dict, String, Float	Adiciona camada de linha conectando pontos
<code>geom_bar(plot, color, width)</code>	Dict, String, Float	Adiciona camada de barras verticais
<code>geom_histogram(plot, color, bins)</code>	Dict, String, Int	Adiciona camada de histograma (distribuição)
<code>geom_boxplot(plot, color, width)</code>	Dict, String, Float	Adiciona camada de boxplot (quartis, mediana)
<code>geom_heatmap(plot, color, cell_size)</code>	Dict, String, Float	Adiciona camada de heatmap (intensidade de cor)
<code>geom_area(plot, color, size)</code>	Dict, String, Float	Adiciona camada de área preenchida sob linha
<code>geom_hline(plot, color, size, yint)</code>	Dict, String, Float, Float	Adiciona linha horizontal em yintercept
<code>geom_vline(plot, color, size, xint)</code>	Dict, String, Float, Float	Adiciona linha vertical em xintercept
<code>geom_abline(plot, color, size, slope, int)</code>	Dict, String, Float, Float, Float	Adiciona linha diagonal $y = slope \cdot x + int$
<code>geom_step(plot, color, size, dir)</code>	Dict, String, Float, String	Adiciona linha em degrau (hv/vh)
<code>scale_x_log10(plot)</code>	Dict	Define eixo X para escala logarítmica base 10
<code>scale_y_log10(plot)</code>	Dict	Define eixo Y para escala logarítmica base 10
<code>filter_data(df, col, value)</code>	DataFrame, String, Float	Filtra DataFrame onde <code>col == value</code>
<code>set_dimensions(plot, w, h)</code>	Dict, Int, Int	Define dimensões do SVG (padrão: 800x600)
<code>set_margins(plot, t, b, l, r)</code>	Dict, Int, Int, Int, Int	Define margens do plot (padrão: 20px)
<code>set_background_color(plot, color)</code>	Dict, String	Define cor de fundo (padrão: white)
<code>set_grid(plot, show_grid)</code>	Dict, Bool	Habilita/desabilita grid (padrão: true)

[title=**Nota:** Faceting descontinuado] As funções `facet_wrap` e `render_facets` foram descontinuadas na v0.8.0. Use `filter_data()` para filtrar DataFrames manualmente e chame `hayplot` para cada grupo.

Nota: PNG export

A função `save_png` requer compilação com a feature `png`: `cargo build --release --features png`. O backend PNG atualmente suporta apenas geometrias básicas (point, line, bar, area).

Cores disponíveis em todas as geometrias:

```
"red"      "blue"  "green"  "yellow"
"magenta"  "cyan"   "black"
```

Pipeline mínimo funcional:

```
1 import("sheep-farm/hayplot", as=gg)
2
3 let df = dataframe({"x": [1.0, 2.0, 3.0], "y": [2.0, 4.0, 6.0]})
4 let svg = gg::hayplot(df, {"x": "x", "y": "y"})
5           |> gg::geom_point("blue", 6.0)
6           |> gg::render_svg
7 write(svg, "out.svg")
```

Listing 8.1: Menor script completo